

SIMATS ENGINEERING

C02-ASSESSMENT TOOL 3

COURSE NAME: Machine Learning

COURSE CODE: ITA0611

5.A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization.

6.A convolutional neural network (CNN) used for image classification produces unstable results across training epochs. Examine potential issues such as improper data augmentation, vanishing gradients, or batch normalization errors. Describe debugging strategies and suggest optimization techniques to stabilize training and improve accuracy.

Code of Conduct:

*I K.ASWINI(192312397) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

ANSWERS

1. Possible Causes

Overfitting

- The tree learns noise and specific patterns from training data.
- Happens when the model is too complex.

Deep Tree Structure

- Very large depth → too many splits.
- Each leaf may represent very few samples → poor generalization.

Noisy / Irrelevant Data

- Incorrect labels or random variations.
- Irrelevant features confuse the model.

Small Sample in Leaves

- If leaf nodes have very few samples, predictions become unreliable.

2. Debugging Steps

Check Model Complexity

- Print tree depth:

```
print(model.get_depth())
```

- If depth is very high → overfitting.

🔍 Compare Train vs Test Accuracy

```
print("Train Accuracy:", train_acc)
print("Test Accuracy:", test_acc)
```

- Big gap → overfitting confirmed.

🔗 Visualize Tree

- Helps identify unnecessary splits:

```
from sklearn.tree import plot_tree
plot_tree(model)
```

🔗 Check Feature Importance

```
print(model.feature_importances_)
```

- Low importance features can be removed.

🔗 Cross-Validation

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=5)
print(scores.mean())
```

- Gives realistic performance estimate.

3. Optimization Methods

Pruning (Most Important)

Pre-pruning (limit growth)

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(
    max_depth=5,
    min_samples_split=10,
    min_samples_leaf=5
)
```

Post-pruning

```
model = DecisionTreeClassifier(ccp_alpha=0.01)
```

Cross-Validation Tuning

Find best parameters:

```
from sklearn.model_selection import GridSearchCV

params = {
    'max_depth': [3, 5, 7],
    'min_samples_split': [2, 5, 10]
}

grid = GridSearchCV(DecisionTreeClassifier(), params, cv=5)
grid.fit(X, y)

print(grid.best_params_)
```

Feature Selection

- Remove irrelevant features:

```
# Example: Select top features
import numpy as np
important = np.argsort(model.feature_importances_) [-3:]
X_new = X[:, important]
```

Handle Noisy Data

- Clean dataset
- Remove outliers
- Use better preprocessing

Use Ensemble Methods (Advanced)

- More robust than a single tree:
 - Random Forest
 - Gradient Boosting

Final Conclusion (Exam Point)

- High training accuracy + low test accuracy → **overfitting**
 - Causes: deep tree, noise, irrelevant features
 - Debugging: check depth, accuracy gap, cross-validation
 - Optimization:
 - **Pruning (pre & post)**
 - **Cross-validation**
 - **Feature selection**
 - **Noise handling**
-

6. This is a **training instability problem** in a CNN—when results fluctuate a lot across epochs instead of improving smoothly.

1. Possible Causes

Improper Data Augmentation

- Too aggressive transformations (rotation, flip, zoom).
- May distort images → model learns wrong patterns.

Vanishing / Exploding Gradients

- Gradients become too small or too large during backpropagation.
- Common in deep networks without proper design.

Batch Normalization Issues

- Incorrect placement or small batch size.
- Causes unstable mean/variance estimation.

Learning Rate Problems

- Too high → loss oscillates.
- Too low → very slow or stuck training.

Poor Weight Initialization

- Bad initialization leads to unstable updates.

2. Debugging Strategies

Monitor Training Curves

- Plot **loss vs epochs**:

```
import matplotlib.pyplot as plt
plt.plot(train_loss)
plt.plot(val_loss)
```

- Fluctuations → instability confirmed.

Check Gradient Values

```
for param in model.parameters():
    print(param.grad)
```

- Very small → vanishing gradients
- Very large → exploding gradients

Verify Data Augmentation

- Temporarily **disable augmentation** and test.
- If stability improves → augmentation is the issue.

Check Learning Rate

- Try smaller values:

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

Evaluate Batch Size

- Small batch → unstable updates
- Try increasing batch size (e.g., 32 → 64)

3. Optimization Techniques

Fix Data Augmentation

- Use moderate transformations:

```
transforms.RandomRotation(10)
transforms.RandomHorizontalFlip()
```

Use Batch Normalization Properly

- Add after convolution layers:

```
nn.Conv2d(...)
nn.BatchNorm2d(...)
nn.ReLU()
```

Gradient Clipping

- Prevent exploding gradients:

```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

Better Activation Functions

- Use **ReLU** / **LeakyReLU** to avoid vanishing gradients:

```
nn.ReLU()
```

Learning Rate Scheduling

- Reduce LR over time:

```
scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=5,
gamma=0.1)
```

Weight Initialization

```
torch.nn.init.xavier_uniform_(layer.weight)
```

Use Dropout (Regularization)

```
nn.Dropout(0.5)
```

Use Advanced Optimizers

- Adam or RMSprop for stability:

```
optimizer = torch.optim.Adam(model.parameters())
```

Final Conclusion (Exam Point)

- Instability occurs due to:
 - Improper augmentation
 - Vanishing/exploding gradients
 - Batch normalization errors
 - Learning rate issues
- Debugging:
 - Monitor loss curves
 - Check gradients
 - Test without augmentation
- Optimization:
 - Proper augmentation
 - Batch normalization
 - Gradient clipping
 - Learning rate tuning & scheduling
 - Better activations and initialization